

8/14/2026

Proyecto 4

Organización de computadores 2026-2

Mateo Botero Vera, Ana Isabella Gómez García
EAFIT

1. Contenido

2. Propósito
3. Interfaz general
4. Arquitectura
5. Programas del proyecto — Mult.asm, Fill.asm
6. Extensiones del curso — SumN.asm, CopyBlock.asm
7. Conclusiones

2. Propósito

El Proyecto 4 tiene como objetivo introducir la programación en lenguaje ensamblador Hack, mediante la implementación de programas que resuelven tareas aritméticas y de control de hardware (teclado y pantalla) usando únicamente las instrucciones nativas de la plataforma Hack: instrucciones de tipo A (@valor) e instrucciones de tipo C (cálculo, destino y salto), sin contar con multiplicación, división ni estructuras de control de alto nivel.

3. Interfaz general

Entradas

- R0, R1, R2 — posiciones de memoria RAM (0, 1 y 2) usadas como parámetros y resultado en Mult.asm
- KBD [24576] — registro de memoria mapeado al teclado
- N — parámetro de tamaño, usado en las extensiones SumN.asm y CopyBlock.asm

Salidas

- R2 — resultado del producto en Mult.asm
- SCREEN [16384–24575] — mapa de bits de la pantalla, 8192 palabras de 16 bits
- R1 u otra posición de memoria destino, según el programa

4. Arquitectura

Todos los programas siguen el mismo patrón general de la plataforma Hack: un ciclo de instrucciones que se ejecuta secuencialmente sobre la ROM, con saltos condicionales (JGT, JEQ, JMP, etc.) que permiten construir bucles y condicionales a partir de comparaciones contra cero. El estado del programa se mantiene en la memoria RAM (variables) y en el registro A/D del procesador.

5. Programas del proyecto

Mult.asm

Comportamiento

Calcula el producto de dos números presentes en R0 y R1, y almacena el resultado en R2, es decir $R2 = R0 \times R1$. Se asume que R0 y R1 son enteros no negativos.

Estrategia

Como el lenguaje Hack no ofrece una instrucción de multiplicación, el producto se calcula por suma repetida: se inicializa R2 en 0 y se suma R0 a R2, R1 veces, usando un contador que se decrementa en cada iteración hasta llegar a cero.

Pseudocódigo

```
R2 = 0
i = R1
while (i > 0):
    R2 = R2 + R0
    i = i - 1
```

Esbozo en ensamblador Hack

```
// R2 = R0 * R1
@R2
M=0
```

```

@R1
D=M
@i
M=D
(L00P)
@i
D=M
@END
D;JLE
@R0
D=M
@R2
M=M+D
@i
M=M-1
@L00P
0;JMP
(END)
@END
0;JMP

```

Componentes/registros utilizados

- Registro A (direccionamiento) y registro D (acumulador)
- Variable auxiliar i como contador de iteraciones
- Instrucciones de salto condicional JLE y JMP

Fill.asm

Comportamiento

Escucha continuamente el estado del teclado (KBD). Si hay una tecla presionada, pinta toda la pantalla de negro; si no hay ninguna tecla presionada, la deja (o la deja) en blanco. El programa corre en un ciclo infinito, respondiendo en tiempo real a la entrada del usuario.

Estrategia

Se usa un bucle externo que revisa permanentemente KBD, y un bucle interno que recorre las 8192 palabras del mapa de memoria de pantalla (SCREEN, direcciones 16384 a 24575) escribiendo en cada una el valor determinado por el estado del teclado (-1 para negro, 0 para blanco).

Pseudocódigo

```

loop:
    color = (KBD != 0) ? -1 : 0
    for addr = SCREEN to SCREEN+8191:
        RAM[addr] = color
    goto loop

```

Componentes/registros utilizados

- Puntero de dirección de pantalla, incrementado en cada iteración del bucle interno
- Constante SCREEN (16384) como base del mapa de bits
- Instrucciones de salto JEQ/JNE y JMP para los bucles anidados

6. Extensiones del curso

SumN.asm

Comportamiento

Calcula la suma de los primeros N números naturales ($1 + 2 + \dots + N$), donde N se recibe en R0 y el resultado se almacena en R1.

Estrategia

Análoga a Mult.asm: se usa un acumulador inicializado en 0 y un contador que recorre los valores desde N hasta 1, sumando el valor del contador al acumulador en cada iteración.

Pseudocódigo

```
R1 = 0
i = R0
while (i > 0):
    R1 = R1 + i
    i = i - 1
```

Componentes/registros utilizados

- Acumulador R1 y contador auxiliar i
- Instrucciones de salto JLE y JMP

CopyBlock.asm

Comportamiento

Copia un bloque de N posiciones consecutivas de memoria desde una dirección de origen hacia una dirección de destino. R0 contiene la dirección de origen, R1 la dirección de destino y R2 el número N de palabras a copiar.

Diseño

- Se usan dos punteros (origen y destino) inicializados a partir de R0 y R1
- En cada iteración se lee la palabra apuntada por el puntero de origen mediante direccionamiento indirecto y se escribe en la posición apuntada por el puntero de destino
- Ambos punteros se incrementan y el contador N se decrementa hasta llegar a cero

Pseudocódigo

```
src = R0
dst = R1
i = R2
while (i > 0):
    RAM[dst] = RAM[src]
    src = src + 1
    dst = dst + 1
    i = i - 1
```

Componentes/registros utilizados

- Punteros auxiliares src y dst, actualizados por direccionamiento indirecto (@src / A=M)
- Contador i e instrucciones de salto JLE y JMP

7. Conclusiones

Los cuatro programas muestran cómo, a partir de un conjunto reducido de instrucciones (aritmética simple, comparación contra cero y saltos), es posible construir bucles, acumuladores y recorridos de memoria equivalentes a

las estructuras de control de lenguajes de alto nivel. Mult.asm y SumN.asm comparten el patrón de acumulación por repetición, mientras que Fill.asm y CopyBlock.asm ilustran el recorrido secuencial de bloques de memoria mediante punteros, sentando las bases para el manejo de memoria dinámica que se formaliza en el Proyecto 5.